

Research Commentary: Weighing the Benefits and Costs of Flexibility in Making
Software: Toward a Contingency Theory of the Determinants of Development Process
Design

Author(s): Robert D. Austin and Lee Devin

Source: *Information Systems Research*, Vol. 20, No. 3, Flexible and Distributed Information
Systems Development (September 2009), pp. 462-477

Published by: INFORMS

Stable URL: <http://www.jstor.org/stable/23015475>

Accessed: 10-11-2016 13:25 UTC

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at
<http://about.jstor.org/terms>



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Information Systems Research*

Research Commentary

Weighing the Benefits and Costs of Flexibility in Making Software: *Toward a Contingency Theory of the Determinants of Development Process Design*

Robert D. Austin

Copenhagen Business School, Frederiksberg, Denmark, and Harvard Business School,
ra.lpf@cbs.dk

Lee Devin

Swarthmore College, Swarthmore, Pennsylvania 19081, ldevin1@swarthmore.edu

In recent years, flexibility has emerged as a divisive issue in discussions about the appropriate design of processes for making software. Partisans in both research and practice argue for and against plan-based (allegedly inflexible) and agile (allegedly too flexible) approaches. The stakes in this debate are high; questions raised about plan-based approaches undermine longstanding claims that those approaches, when realized, represent maturity of practice. In this commentary, we call for research programs that will move beyond partisan disagreement to a more nuanced discussion, one that takes into account both benefits and costs of flexibility. Key to such programs will be the development of a robust contingency framework for deciding when (in what conditions) plan-based and agile methods should be used. We develop a basic contingency framework in this paper, one that models the benefit/cost economics described in narratives about the transition from craft to industrial production of physical products. We use this framework to demonstrate the power of even a simple model to help us accomplish three objectives: (1) to refocus discussions about the appropriate design of software development processes, concentrating on when to use particular approaches and how they might be usefully combined; (2) to suggest and guide a trajectory of research that can support and enrich this discussion; and (3) to suggest a technology-based explanation for the emergence of agile development at this point in history. Although we are not the first to argue in favor of a contingency perspective, we show that there remain many opportunities for information systems (IS) research to have a major impact on practice in this area.

Key words: software development; software methodologies; work design; process design

History: Brian Fitzgerald, Senior Editor and Associate Editor. This paper was received on February 1, 2008, and was with the authors 5½ months for 2 revisions.

Introduction and Literature Review

In recent years, flexibility has emerged as a divisive issue in discussions about the appropriate design of processes for making software (Agerfalk and Fitzgerald 2006). Some argue that plan-based approaches, although widely used, are too bureaucratic and inflexible for a rapidly changing modern world (e.g., Beck 1999). Others express concerns about efficiency and the quality of outputs from flexible approaches (e.g., Parnas 2006). Still others occupy a middle ground, urging different approaches for different circumstances (e.g., Boehm and Turner

2003, Harris et al. 2009). The issues involved in these discussions are best considered in historical context.

Ad hoc software development (ca. the 1960s and before) left design and management of processes to individual programmers whose personalized tricks of the trade (Dijkstra 1968) often resisted standardization. Processes and their management varied widely. Products were customized for a particular purpose even when automating standard functionality. Such widely diverse products posed quality assurance, maintenance, and productivity challenges. For exam-

ple, repairing software defects meant confronting spaghetti code (Dijkstra 1968).

Plan-based development arose to address the shortcomings of *ad hoc* development. Royce (1970) proposed an influential model that has become known as the “waterfall” (although Royce never called it that). He described a *sequential* process that moves a product from abstraction to physical detail in stages, calling on developers to create representations (documents) that specify the features a product will need, then to develop those specifications in stages of increasing detail until they become increasingly code-like and, finally, actual code. Related models varied in specifics but shared an emphasis on planning to anticipate changing conditions and a structure composed of separate stages through which a product moved toward completion (though not always only in a straight line).

Many proponents of plan-based models drew parallels between their efforts to better organize software production and the historic transition from craft to industrial production of physical products. Some called for a “software industrial revolution,” a shift to “a more product-centric paradigm in which progress is measured by the accretion of standard, interchangeable, reusable components” (Cox 1990, p. 25). This vision calls for software to be “built by assembling components and...[to become] functional with little effort since the components are already developed, tested, and matured” (Fröberg 2002, p. 59). Making software this way requires planning to standardize component interfaces and other structures, and reduce idiosyncratic interdependence between and within components. Elimination of such interdependencies, and the grander objective of “transforming programming...into an organizational enterprise like manufacturing” (Cox 1990, p. 28), motivated efforts to standardize instruction formats (Bohm and Jacopini 1966, Dijkstra 1968, Dahl et al. 1972, Knuth 1974), objects (Dahl and Nygaard 1967, Kay 1993), components (Wirth and Gutknecht 1992), and off-the-shelf software packages (Brooks 1987, Voas 1998). More standardized and structured development processes promised improved quality, maintainability, and efficiency (productivity). Many advantages have now been widely realized but the larger vision remains

aspirational, having now evolved into a conversation about “service-oriented architecture.” There are claims, however, that the larger vision has been realized in particular contexts (Fröberg 2002, Marwaha et al. 2006).

Eventually (beginning especially in the late 1990s), plan-based models attracted critics who disputed the validity of industrial revolution parallels and pointed out problems in supposedly state-of-the-art methods (Beck 1999, Highsmith 1999). Many complaints centered on the alleged *inflexibility* of plan-based approaches. Because the stages of such approaches unfold over time, they can in turbulent environments produce solutions to problems that have already changed. Even in stable environments, software’s intangibility and inscrutability to laypersons make it difficult for end users to explain in advance what they need: “I...assert that it is really impossible for a client, even working with a software engineer, to specify completely, precisely, and correctly the exact requirements of a modern software product before trying some version of the product” (Brooks 1987, p. 17). Critics of plan-based approaches argue that to be sufficiently flexible, especially in increasingly turbulent environments, a software process needs to revisit specifications and refine requirements even late in development. Supporters of plan-based approaches, long aware of this need, propose to meet it with a feature already built into their models: feedback loops that allow projects to revisit earlier development stages when necessary (Mills 1971, Osterweil 1987, Jacobson et al. 1999, George 2000).

Advocates of *agile* development go further. They propose intensely iterative processes—cyclical structures that right from the start move quickly and frequently to code, creating new versions of the software daily or even more often and replanning each time to incorporate requirement changes, adjust to unanticipated problems, and include new ideas generated by the work itself. Such frequent iteration and the move to code so early in a project threatens the values integral to plan-based methods, especially planning before doing, progression toward completion in identifiable stages, and reliance on detailed documentation (no time for that amid rapid iteration). The differences establish, in the minds of many, boundaries between “two ostensibly conflicting

approaches to software development [competing] for hegemony” (Boehm and Turner 2003, p. xix). Highsmith (2002) reports an exchange between agile proponent Kent Beck and skeptic Alan Cooper that captures the essence of the disagreement:

Cooper: I’m worried about the fact that...you [agile developers] don’t do any design.
Beck: We do design for a couple of hours each day.
Cooper: That’s what I’m concerned about! You don’t do any design!

As Cooper made more explicit later in the same exchange, to him “design” meant activity carried out before construction begins.

At the center of this debate is a disagreement about the amount of flexibility a development process needs. Let us define *flexibility* as that characteristic of a process for making software that allows it to easily produce novel output as demanded by the environment in which the software functions. By “novel,” we mean simply that the output is different than it was before, changed to address a new or newly perceived demand for additional or different functionality. “Novel” does not by itself imply value, although novelty is usually generated with the intention of creating value, nor does novelty imply *radical* departure from a product’s past form; both incremental and radical changes count as output novelty in the way we are using this terminology.¹

When the world changes in a way that requires different functionality from a software product being built, a flexible development process allows adjustment of the product to reflect the changed world. A less flexible process *might* allow such an adjustment but has a limited capacity to incorporate change: It could be limited in the frequency of changes it can handle, or in the magnitude of each individual change or both. For Kent Beck, the environment is full of important and frequent changes, large and small, so iteration that includes replanning (“design”) must happen often (“each day”).²

However, for Alan Cooper, such frequency of iteration and replanning, and the resulting abundance of product changes, are unnecessary and unwise—by spending more time on design before construction begins, developers could better understand requirements and anticipate many problems, eliminating much of what Beck perceives as demand for change in the environment. For plan-based advocates, too much novelty of output from a development process is a symptom of insufficient planning discipline. Moving to code so early and often recalls *ad hoc* development, when absence of forethought about standardization led to an abundance of idiosyncratic coding novelty, which in turn drove demand for more coding novelty (Parnas 2006).

Empirical findings have not resolved this disagreement. We know that some commercial software companies, under time-to-market pressures, avoid plan-based approaches and adopt practices that resemble agile development (Iansiti and MacCormack 1997, MacCormack et al. 2001). In a similar vein, Eisenhardt and Tabrizi (1995) point to changes in the computer industry (not just in software) that have led to adoption of “accelerated adaptive processes,” which share characteristics with agile approaches. Within the broader research on product development, however, we find a parallel debate, also unresolved, about the relative merits of plan-based “stage gate” models (Cooper 1986, 1988; Clark and Fujimoto 1991) versus iterative models (e.g., Schrage 2000, Thomke 2003).³ Banker and Kemerer (1989) and Banker and Slaughter (1997) show that firms practicing plan-based methods realize economies of scale in software development and maintenance.

Methods that emphasize iterative structure in software development are not new. Antecedents of the agile movement include the “spiral model” (Boehm 1988), “rapid application development” (Martin 1991), and “joint application design” (Wood and Silver 1995), all of which had some following but none of

¹ In adopting this terminology, we follow the example of Nambisan (2003) and link this discussion of software development to broader literatures on new product development (e.g., Clark and Fujimoto 1991), business innovation (e.g., Christensen 1997), and the psychology of creativity (e.g., Amabile 1996). Those literatures adopt definitions of creative or innovative outputs that ascribe two main characteristics to such outputs: (1) novelty and (2) value.

² It must also happen very quickly: Beck (2004) calls for a “ten minute build,” in his words: “automatically build the whole system and run all the tests in ten minutes” (p. 49).

³ Nambisan (2003) has suggested that information systems should serve as a “reference discipline” for new product development, rather than the reverse.

which has become mainstream practice. Ideas about agility in software development can also be traced to earlier ideas about “lean” manufacturing (Krafcik 1988, Womack et al. 1990, Clark and Fujimoto 1991, MacDuffie 1995, Shah and Ward 2003). Many principles, such as elimination of needless activities and minimization of batch sizes (i.e., more frequent production of changed outputs), appear common to the two domains. Elimination of buffer inventory, which adds complexity and lead time in manufacturing (Krafcik 1988), and efforts to solve problems near (in both space and time) to where they happen (Spear and Bowen 1999), are both central to lean manufacturing, and they parallel agile software development practices that reduce complexity and respond rapidly to problems. Companies occasionally use lean manufacturing as the model for initiatives to improve software development (see Staats and Upton 2007 for an example from the information technology service firm Wipro). However, there is healthy debate about the need for contingency in applying “lean” principles to manufacturing (Hopp and Spearman 2004) and a significant lack of clarity about what “lean” means when applied to software development (Staats and Upton 2007). In the end, neither historical precedent nor manufacturing parallel provides satisfactory guidance for design of software development processes.

Part of the difficulty in resolving these disagreements, as Agerfalk and Fitzgerald (2006) note, is that “practice is ahead of research in this area” and that “the fundamental underpinnings of agile methods need to be better conceptualized and theorized” (p. 27). Although we hope to make headway in that direction in this paper, we acknowledge the difficulty of defining “agile.”⁴ Practices vary and proponents differ on which practices must be adopted for the overall approach to be called “agile” (Conboy 2009). Rather than propose a definition that will fail to satisfy some agile advocates, we focus more narrowly in this paper on the fundamental issue of *process structure*, the extent to which a development process relies on planning before doing and sequential progress through distinct stages as opposed to frequent iteration that blurs

distinctions between stages.⁵ Although process structure does not encompass all the differences between plan-based and agile approaches, it plays a central role in discussions about development process design. Process structure also relates closely to the issue of flexibility because the reason for iteration in software development is to create novel output (a new version of the software different from the previous version).

To determine the appropriate process structure for development in a given context, we propose a contingency perspective. We are not, of course, the first to suggest that the best answer to questions about design of development processes might be “It depends.” A common argument suggests that agile approaches are best-suited to turbulent environments and plan-based approaches the best-suited to stable environments (e.g., Highsmith 2002, Agerfalk and Fitzgerald 2006). Boehm and Turner (2003) propose “a risk-based approach to aid in balancing agility and discipline” (p. xix) that recommends agile approaches when speed and flexibility are most important and plan-based approaches when avoiding risk is the highest priority. Paulk (2002) suggests that questions about the best methods to employ in different contexts be answered empirically. Slaughter et al. (2006) conducted such an empirical study and concluded

⁴ The most widely accepted statement of the shared characteristics of agile approaches is probably the “Manifesto for Agile Software Development,” which expresses a consensus developed in 2001 by agile thought leaders; see Beck et al. (2001).

⁵ This distinction in terms of process structure could be related to the distinctions in terms of formality in a development process, but they are not the same thing. Formality usually refers to the “use of a formal logical calculus which is proven sound with respect to a complete mathematical semantics of the specification language” (Sannella 2008, Sannella and Tarlecki 1997, p. 1). Plan-based methods have more often been associated with this kind of formality, and advocates of formal and plan-based methods often share an objective to make software development more scientific. However, many plan-based methods would not qualify as provably sound. Moreover, it is possible to imagine that some iterative methods might qualify equally well. “Formality,” used more loosely, often suggests process rigor and advocates of agile methods have been unwilling to concede that their methods are less rigorous (though this is clearly sometimes in dispute). We define our process structure distinction more narrowly in order to avoid the definitional difficulties of words such as “formal” and “agile.” Differences in process structure, as we define it, hinge simply on the frequency and rapidity of iteration and on the other noticeable characteristics of a process that follow from frequency and rapidity of iteration, such as how early in the recommended process code begins to be produced and how discernibly separate are the stages of development (analysis, design, etc.).

that companies should align software processes with their business strategies, choosing for example flexible designs when they employ differentiation-based strategies. The framework we propose is consistent with each of these contingency stories and also some others, but its generality and the manner of its derivation offer advantages of versatility and interpretability that will soon become clear.

As a basis for our contingency framework, we put forward the simple, general, and (we think) uncontroversial assumption that iteration, which creates novelty of output (product changes) and thus contributes flexibility to a development process, has both *benefits* and *costs*. Decisions about process structures, then, should maximize the difference between benefit and cost in a given set of conditions. Without yet instantiating our analysis with examples of specific benefits (e.g., better execution of differentiation strategies; see Slaughter et al. 2006) or specific costs (e.g., great likelihood that risks will materialize; see Boehm and Turner 2003), we consider how the benefit-cost comparison changes as the structure of the development process changes. The resulting contingency-based framework is compatible with a variety of specific benefit and cost sources and, thus, potentially provides insight into a wide range of circumstances. The framework suggests a technology-based explanation for the emergence of agile development at this point in history, and calls into question the idea that industrialization is necessarily the “mature” endpoint of evolution for making processes. It also suggests a trajectory for future software development research that could lead to a more nuanced discussion than the “strident” (Boehm and Turner 2003, p. xix) debates now current, and offers guidance on how different approaches might be usefully combined.

To develop this framework, it will be useful to consider in greater detail the arguments that depict the evolution of plan-based approaches in terms that parallel the historic transition from craft to industrial production of physical products. Let us treat software making as a special case of making-in-general and step through arguments about transition from craft to industry, taking care to account for the benefits and costs of different ways of arranging production.

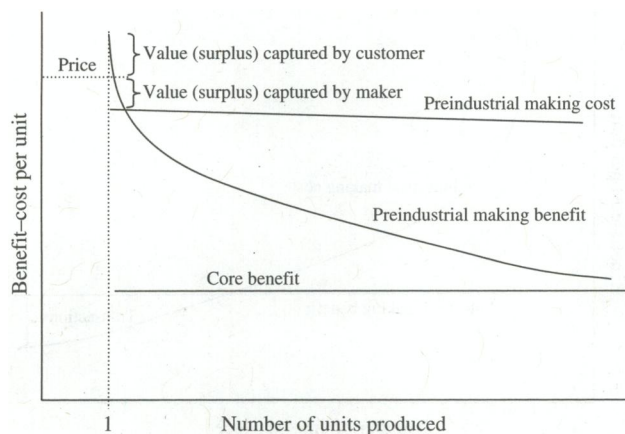
Pre-Industrial Making

Consider a medieval blacksmith who has no ideas about interchangeable parts, standardized work processes, or economies of scale. He makes things, say knives, to order. His work accommodates variation in inputs and accepts interdependence among his materials and methods. One batch of charcoal burns hotter than another; one load of ore has greater or lesser iron content and makes a different bloom of steel than another. These differences mandate differences throughout his processes: in furnace time, in forging, in annealing and tempering, and more. Each knife the blacksmith makes is unique, and it never occurs to him that it could be otherwise.

Pre-industrial making generated three kinds of cost: (1) *reconfiguration cost*, the effort and resources required to arrange and rearrange equipment and materials (Keller and Keller 1996); (2) *exploration cost*, the effort and resources lost when something tried did not work well enough; and (3) *production cost*, the effort and resources consumed in making each part of the sold product. For example, adjusting the fire to account for differences in the iron content of a new batch of ore incurs reconfiguration cost; recycling an incorrectly tempered blade that broke during assembly incurs exploration cost; and the work and materials used to make a saleable knife, independent of reconfiguration and exploration, combine to incur production cost. The sum of these costs is *iteration cost*, the cost of creating a novel product, i.e., a knife that is different from the one that came before it.

For the pre-industrial maker, the work of reconfiguring and exploring happened every time he made a knife. Customer preferences, material properties, his own day-to-day performance—to manage the variations in all of these constituted his expertise. No matter how many knives he made, no matter how similar some might be to others, the maker expected to incur these costs for each one because the differences in materials and the accumulation of differences in the outcomes of uncertain processes required constant adjustments. Thus, the maker expected the average

Figure 1 How Transactions Happen in Pre-industrial Making



cost of making a knife to remain roughly the same no matter how many he made.⁶

Pre-industrial making generated two kinds of benefit: (1) *novelty benefit*, advantages attributable specifically to the novel characteristics of a product, often (but not limited to) benefits from features of the product tailored to a specific customer; and (2) *core benefit*, advantages attributable to features of the product that are common to all such products and available to all customers, such as the nominal ability of a knife to cut meat. A knife made for a man with large, powerful hands might work for him in ways that few others could enjoy (novelty benefit). Wielded by a man with smaller hands, the same knife might not work as well but could still be of use (core benefit).

Figure 1 shows the relationship between pre-industrial benefits and costs and how transactions could occur. In this pre-industrial world, transactions usually involve a single customer placing an order for a unique product. The product has benefit, both novelty and core, to that individual in excess of its rather high cost; but the unique product made for that

customer is not useful enough to other customers to command a price that exceeds the cost of making an additional similar unit. The maker sells one unit at a price between the cost of making and the benefit to a particular customer. Standardized products do not arise naturally from this process. Costs and thus prices are high. Novelty benefit matters a lot; without it, no transaction would occur.

This pre-industrial making process, which equates to *ad hoc* development according to the software industrialization narrative, is flexible. It must be flexible in order to handle the uncontrolled sources of variation within the process, such as different iron content in batches of steel to which the blacksmith must adjust, or nonstandard programming interfaces to which the developer must adjust. With this flexibility the blacksmith can uniquely tailor a product to make it especially valuable for the intended customer. However, flexibility comes at a very high cost and that cost motivates the search for a better way.

Industrial Making

When Eli Whitney offered to manufacture 15,000 muskets for the U.S. government, he had only recently seen a musket up close (Mirsky and Nevins 1952). He chose to make muskets after deciding that only the government had the resources he needed to implement his as-yet-untried “interchangeable system.”

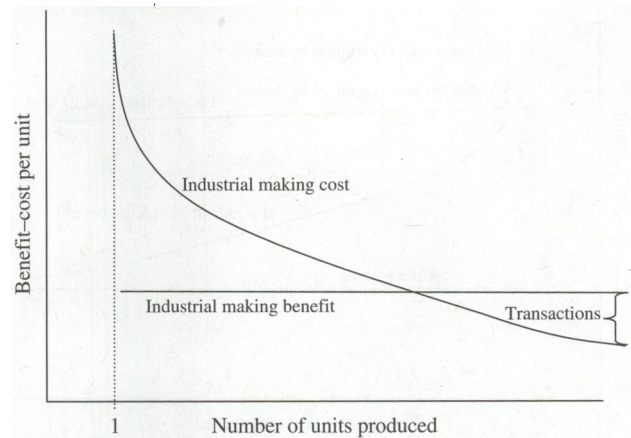
Whitney analyzed a musket into its parts and designed a machine to make each part. Parts made this way achieved unprecedented accuracy in shape and dimensions; they were *interchangeable*. Later, Henry Ford demonstrated the economic advantages of this approach by extending it to the concept of “mass production.” By standardizing his product and its parts and putting them together on a moving assembly line, he increased the number of Model Ts he could make in a year from 78,440 in 1911–1912 to 730,041 in 1916–1917 (Nevins 1954). Frederick Taylor applied to workers themselves the ideas Whitney and Ford used for products and parts. He analyzed jobs into their smallest gestures and timed each one, suggested improvements in arrangements and individual movements, and eventually arrived at “the one best way,” a standard process specification printed on a rate card (Kanigel 1997).

⁶ This assumes that the blacksmith was already an expert, that he had already moved down the learning curve associated with his craft. It is possible that the average cost of making more than one similar item might decline slightly, as there are arguably some economies that can be claimed in this work, even in such a highly variable world. To accommodate this possibility, the figures in this paper depict pre-industrial making costs that decrease slightly.

The changes that accompanied this revolution made the structure of work more *sequential* than it had been. To achieve interchangeability of musket parts or to create the productivity gains of Ford's factory, the product had to be specified in advance and in exacting detail. To realize added efficiencies, process designers preconceived every aspect of the actual making. Design separated from production. The idea behind this separation was not to avoid or reduce the costs of reconfiguration and exploration (the required precision of prespecification probably dictated that these costs increase somewhat, in fact), but rather to extract them from the midst of making and place them at the front of the process, transforming them into what we now call "product development" and "process engineering." Placed at the beginning of a sequential process, costs associated with reconfiguration and exploration could be shared across many units, producing economies of scale.⁷

If we consider the benefit and cost characteristics of industrial making, we notice that this way of working, conceived in its simplest form, does not provide novelty benefit, only core benefit ("Any customer can have a car painted any color that he wants so long as it is black⁸"). Reconfiguration and exploration costs occur once at the beginning of the process, in the design stage, and then the manufacture of each item generates only planned production costs. Figure 2 depicts this situation. In contrast with pre-industrial making, the expected benefit from industrial making remains constant (at the core benefit level) while the cost per unit falls and approaches production cost as the number made increases. Novelty benefits (such as benefits from customizing a product for a specific customer) are not available, so we must make enough of each product to drop unit cost below the price customers will pay for a product that offers only core

Figure 2 How Transactions Happen in Industrial Making



benefit.⁹ When we make a lot of product, costs and thus prices can be low.

Compared to pre-industrial making, this industrial making process has less flexibility, although it is capable of planned variety and some unplanned novelty, whether in manufacturing or software development. The Ford Model T came only in black but consumers could choose between side lamps from Corcoran, Victor, or E&J (variety). To address product problems, react to supply issues, or improve efficiencies, Ford's staff often changed parts and adjusted processes in the middle of a production run (novelty).¹⁰ In the mid-1990s, a Ford assembly line had the planned capability to build eight million different versions of the Taurus automobile (variety), not taking into account paint color (more variety); unplanned part changes within production runs (novelty) were accomplished via "engineering change requests" (Austin 2001). The ability of an industrial process to generate variety and some novelty is, of course, vital to the arguments that draw parallels between industrialization and software development

⁷ Efficiencies from industrial making include savings that result from catching quality problems early in a separate design stage; these efficiencies are more often cited in justifications of plan-based software development than are the pure production economies (e.g., shared overhead costs) that come to mind when contemplating mass production. Either (or, more likely, both) might provide a rationale for industrial making in a given setting.

⁸ Refers to the Model-T, mass-produced only in the color black (Ford 1922, p. 72).

⁹ Individual customers might experience greater or lesser benefits due to their accidentally advantageous compatibility with the standard product (for example, a hand that just happened to be sized to fit the generic product). Core benefit, which can lead to sales beyond the numbers of a few accidentally compatible customers, is the benefit-related quantity that determines the economic viability of this standardized process.

¹⁰ For instance, Ford Staff changed the cylinder casting at about unit 300,000; see McCalley (1994) for a detailed account of changes made in production.

maturity, because software development rarely (if ever) produces many identical outputs¹¹ and there is always some need to adjust. This need to adjust accounts for the loops back through earlier stages of development built into plan-based models. A key requirement of adjustments made in this way, however, is that they do not negate scale economies or other advantages of industrial making.

Thus, there is a limit to the flexibility that industrial making can provide. Too many novel outputs or adjustments that are too great in magnitude can undermine the advantages provided by an industrial process structure. Naturally, then, managers of such processes make efforts to avoid incurring repeated upfront costs by anticipating future product needs and accounting for them in the initial design; by conceiving different process outcomes, not as unplanned novelty, but as planned variety; and by doing extensive design before construction begins.

Post-Industrial Making

The ideas and technologies that enabled the Industrial Revolution reduced unit cost by extracting reconfiguration and exploration costs from the making of every unit and relocating them to the beginning of a making process, so that their impacts on eventual products

could be reduced by scale efficiencies. Reducing cost this way sacrificed some ability to produce novel outputs, a sacrifice that was more than outweighed by the resulting dramatic reduction in unit cost.

In recent years, however, the possibility of another option for reducing unit cost has appeared, one that we call “post-industrial making.” Instead of seeking scale efficiencies, post-industrial making lowers reconfiguration and exploration costs directly, often by using information technology (Schrage 2000, Thomke 2003, Austin and Devin 2003). If costs of reconfiguration and exploration in a particular context make an iterative process structure too expensive, then why not reconfigure and explore through simulation, in the virtual, instead of the physical, world? Rather than unbend and recycle physical metal, for example, why not rearrange and retry electronic bits and bytes, which can be done more cheaply (once initial investments in the technology are complete, at least)? The advantage of this approach, if it is feasible, is that it realizes efficiencies without sacrificing the ability to produce novel outputs (flexibility).

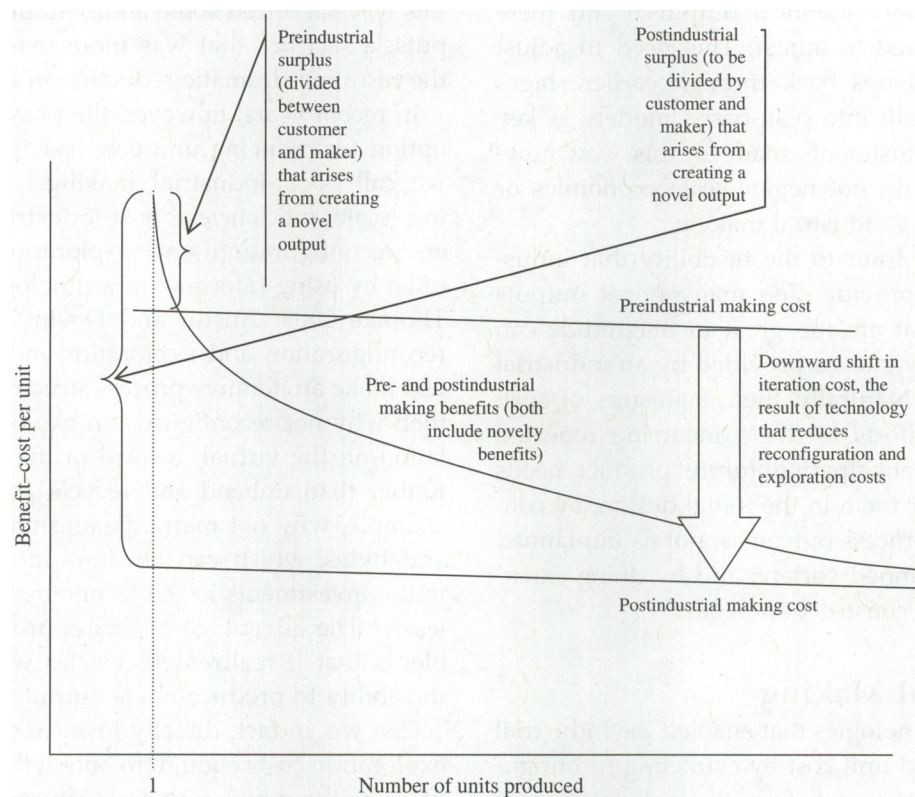
Can we, in fact, directly lower reconfiguration and exploration costs enough to obtain the benefits of flexibility without high unit costs returning to offset those benefits? Figure 3 depicts the “third way” to organize a making process suggested by such a possibility. Post-industrial making drives down the costs of producing novel outputs and at the same time retains novelty benefits forsaken by industrial making. Thus, it creates much more value from each iteration than its ancestor pre-industrial making ever could. Post-industrial making resembles pre-industrial making in some ways (its iterative structure, for example) and the resemblance might lead those steeped in industrial making (plan-based development) to see post-industrial making as *identical* to pre-industrial making (*ad hoc* development), and thus as a step backward. This would be a mistake. Post-industrial making is a different way to organize production. Of course, it can still be debated whether this different way is what we are observing in the emergence of agile development or, indeed, whether post-industrial making is at all feasible. As we have introduced it here, it is a theoretical possibility, nothing more.

This theoretical possibility, however, also serves as a basis for a contingency perspective. Two quantities are particularly important: novelty benefit and the

¹¹ Slaughter et al. (2006) provide a specific sense of the nature of scale economies in software development, an activity in which “the volume of production (number of copies produced) is not as relevant”:

Although the ‘production process’ in software development is knowledge work, it can be quite repetitive (Kemerer 1992). For example, computer-aided software engineering tools, programming languages, and development methodologies can be used in a repetitive way across projects Thus scale economies can occur within a large software project or across a set of related projects because developers can learn and become more efficient as they repeat one or more tasks. Economies can also occur because investments in overhead activities that facilitate project performance (such as project management, configuration management, testing, and quality control) can be spread over a larger base (Slaughter et al. 1998). For example, the cost of setting up sophisticated testing environments and databases can be recovered in a sufficiently large project or set of projects.

Thus, if we follow the logic of software industrialization, the many similar units that create scale economies in software development are most likely to be subparts of a project or a set of projects with similar content.

Figure 3 How Transactions Happen in Post-industrial Making

sum of reconfiguration and exploration cost, which we can label *novelty cost*. When novelty benefit is high relative to core benefit there is a possibility that post-industrial making could create a benefit/cost surplus greater than that from industrial making. For the possibility to become economically feasible, novelty cost must also be low relative to production cost. As Figure 4 indicates, highly iterative—“agile”—process structures arise appropriately only in a special set of conditions. Novelty benefit must be adequately high and novelty cost must be adequately low.

This description of post-industrial making suggests a technology-based explanation for the appearance of agile development at this point in history. The transition from craft to industrial production of physical products was made possible by technologies (most notably, precision machine tools) that permitted parts specifications derived in a separate design stage to be realized reliably, thus rendering parts interchangeable. People like Whitney, Ford, and Taylor made a revolution but none of it would have been possi-

ble without the appearance at the right time of key enabling technologies.

The transition of software development from *ad hoc* to plan-based development involved no similar

Figure 4 Conditions in Which Agile Process Structures Can Appropriately Arise

Benefit	H	You can get more benefit from novel outputs than is offset by the cost of producing them... POST-INDUSTRIAL (AGILE) PROCESS	You can get benefit from novel outcomes, but the benefit is more than offset by the cost of producing those novel outcomes... INDUSTRIAL PROCESS
	L	You could design a process to produce novelty cheaply, but there's too little benefit in it... INDUSTRIAL PROCESS	It's not feasible to produce novelty because it costs too much and there's little benefit in it anyway... INDUSTRIAL PROCESS
		L	H
		Novelty cost(reconfig. + exploration cost)	

technological dependency. Software has been, from the beginning, a malleable medium that motivated programmers could form precisely. Evolution of software processes parallel to industrial processes awaited mainly the development of new methods (e.g., “structured programming”) (see Dijkstra 1968, 1969; Knuth 1974) and the (not always easy) brokering of agreements on standards. Some new technologies helped (e.g., object-oriented languages) but those appeared quickly, at least compared to the rate at which machine tools and other mechanical technologies came along to enable Whitney, Ford, and Taylor. Software making moved rapidly toward industrial process, accomplishing in a few years what had taken physical making decades or centuries.

Given this history, discussions about software development process design, especially arguments based on the idea of progress toward greater maturity of development processes, prompt an important question: *Do industrial methods represent the “mature” endpoint in the evolution of making process structure, or is there another process structure into which industrial structure might evolve?*

The contingency perspective we propose suggests an answer to this question that reinforces the legitimacy of agile development: Just as key technologies made a transition from pre-industrial to industrial making possible, a different set of key technologies now enable (for some kinds of making) another transition, from industrial to post-industrial. It is clear that new technologies are helping to manage sources of high novelty costs. Rapid prototyping and automated build technologies make it easier to rapidly assemble a new software version (reducing reconfiguration cost). Automated testing and simulation make assessment of problems with the new version quicker and easier (reducing exploration cost) and sophisticated version control systems allow developers to roll back “mistakes” quickly, reliably, and selectively (also reducing exploration cost). Thanks to these and similar innovations, the special circumstances required for agile process structures might be becoming more common.

Apparently, similar developments in the world of physical product development lend credence to this interpretation. Thomke (2003) has enumerated ways that technology is increasingly applied to reduce the cost of experimentation and product prototyping in

physical product development, drug discovery, and elsewhere, leading to more iterative development processes. In a case study about the design of America’s Cup racing boats, Iansiti and McCormack (1996) show how the New Zealand team’s use of simulation on high-powered workstations collocated and integrated with rapid fabrication equipment allowed a previously plan-based process, keel making, to become much more iterative and much more productive of valuable novelty (the New Zealand team easily defeated the competition). A widening array of technologies such as 3D printing, electron beam melting, fused deposition modeling, laminated object manufacturing, selective laser sintering, and stereolithography, technologies that turn design changes cheaply and almost immediately into physical objects, lead us to believe this trend will continue.

Refinement of software product design methods inspired by physical product design methods may add momentum to shifts toward more iterative, agile processes. For example, “product line” approaches to software engineering (see Northrop et al. 2007) argue for the proactive arrangement of software products into lines that share assemblies, parts, and other components. Once a line-based product architecture exists, new benefits can be achieved by cheaply assembling existing components in new configurations (reconfiguration cost has been reduced). Even when novelty beyond what would be possible by assembling predesigned components is required of a software product, product line architectures can help enable more iterative development by making it possible to arrive, quickly and cheaply, at a starting point from which the novel functionality in a product can be developed. This example shows how process rigor and organization might be further instilled in a post-industrial making process once the shift to such processes is enabled by the emergence of new technologies, methods, and architectures. It also highlights the conceptual advantages of a contingency perspective: whereas the software product line approach is often described as a way of *industrializing* software development, we see from within a contingency framework that the approach can also be considered a way of combining industrial (planning to set up an initial architecture) and post-industrial (subsequent iterative development processes) methods.

As we have described it here, the shift from industrial to post-industrial making, like the earlier shift from pre-industrial to industrial making, is technology driven and not market driven. Before this shift, the business environment did contain latent demand for novel (e.g., customized) products at lower prices, just as the Industrial Revolution environment contained latent demand for cheap standardized products. Until the requisite technologies came into being—machine tools and so on in the Industrial Revolution, digital simulation, and prototyping technologies in the Post-Industrial Revolution—the latent demand had to remain unsatisfied. Without these requisite technologies, costs remained too high to allow suppliers to make a profit while providing products at low enough prices to tap the latent demand.

Specific Sources of Benefit and Cost of Novelty

Our general framework can be instantiated with specific sources of benefit and cost to connect its implications to practice and inform debates about

the feasibility of agile practices. Categories of specific benefits and costs include (see Table 1 for a summary):

Novelty Benefit

Originality valued for its own sake—In some contexts, there is perceived value in original expression. Cox (1990) reports this statement from one of Apple's best programmers: "I would no more reuse code than Ernest Hemingway would have reused other people's paragraphs" (p. 26). This is not a very businesslike attitude, but it can apparently influence practice. Now and then, originality of expression for its own sake translates into aesthetic characteristics attractive to customers (Austin 2008). Novelty valued for its own sake by an artful programmer, for example, might result in an elegant system feature that customers would pay for. When this happens, the novelty benefit has evolved out of the "originality for its own sake" category and moved to a new category that we will call "potential for business differentiation."

Potential for business differentiation—Slaughter et al. (2006) show how market opportunities for differentiation and a firm's inclination toward differentiation

Table 1 Specific Sources of Novelty Benefit and Cost

Sources of novelty benefit (Reasons why product changes might be valued)	Sources of novelty cost (Reasons why changing the product might be costly)
Originality valued for its own sake Perceived elegance or artistry in a changed product or system	Lack of "pliability" in materials, process, equipment, or other means of production High cost to rearrange production in order to make something different (e.g., need to buy new equipment, reconfigure a factory, retrain workers)
Potential for business differentiation - Strategic differentiation (firm level) - Brand differentiation - Product differentiation	High cost of time or materials consumed in creating something new - Cost of nonuseful prototypes or practice runs - Recycle, restock, or scrap costs (precious time or materials "wasted")
Difficulty discerning requirements <i>ex ante</i> (changes take into account requirements discovered in process, over time) - Requirements difficult to see, know, or understand in advance - Requirements become better understood over time - Users know their requirements but have trouble communicating them to developers	Coordination burden - Large organization or product (greater scope of follow-on changes necessary after an intended change) - Complexity of organization or product (more discrete subparts to be potentially affected by changes) - Product integrity (tight functional coupling or absence of modularity between components) - Geographically distributed team (difficult or costly to communicate changes and assess follow-on implications across distance)
Variety in requirements across users (customization for different users) Many users need something different (many customer segments); product changes satisfy needs of additional customers	Context-dependent exploration costs - Serious consequences (e.g., loss of life, very great expense) when a prototype, pilot, or experiment "fails" - Testing changes can be costly even if they can be made inexpensively
Technology turbulence Requirements change often because business environment changes rapidly (due to technology advances, new entrants, disruptive business models, etc.); product changes track changing business requirements	

strategies can lead to benefits from flexible development processes. Flexible processes help companies differentiate themselves, for example, by enabling rapid changes in products that competitors cannot match. Contexts vary in the degree to which they offer opportunities for strategic differentiation of companies, brands, or products, depending on a wide range of factors such as market structure (fragmented or not), current choices facing customers (few or many), customer expectations about product performance (purely functional or also aesthetic), etc.

Difficulty discerning requirements ex ante—If customer requirements for a software product are difficult to see and understand in advance and must therefore be discovered during the process of building a system (Brooks 1987), then a development process may benefit from flexibility. The usual description of this problem involves a user who cannot imagine what a new system might do and a developer who does not understand the user's needs; flexibility in the development process can help bridge this communication gap as a software project unfolds.

Variety in requirements across users (customization for different users)—Franke and Von Hippel (2003) develop a “calibrated coefficient of heterogeneity” (H_c), the ratio of (1) the number of different products required to satisfy all customers to (2) the number of customers, to characterize the potential to realize benefits from customizing products in a given customer base. If one product will satisfy every customer, then H_c is close to zero; if, on the other hand, every customer needs a different product to be satisfied, then $H_c = 1$. The nearer H_c approaches one, the more potential novelty benefit we can infer. Conversely, as H_c decreases, the potential novelty benefit decreases because we need fewer different outcomes—less novelty—to satisfy all customers.

Environmental turbulence—A rapidly changing competitive environment is the most commonly cited rationale for a flexible approach. A firm might need novel output to respond to moves by competitors, to keep up with shifting customer expectations, to react to rapid technological advances, or to accommodate other changes in the competitive environment.

Novelty Cost

Lack of “pliability” of materials, process, equipment, or other means of production—Some materials, processes,

and equipment are difficult to reconfigure and some materials are costly to recycle or throw away. Component reordering and recycling or equipment retooling and replacement, for example, can be very expensive. Means of production that are not “pliable” (easy to reconfigure) increase the need for planning because they make iteration expensive. Software development, however, uses very pliable materials. As Slaughter et al. (2006) observe, “software is intangible and more easily adapted than a physical product” and “software processes may be more adaptable than manufacturing processes bound by machinery, raw materials, and physical plants” (p. 915). This provides a stronger rationale for iterative process structures in software development than in physical settings, and raises questions about whether justifications of plan-based methods in physical settings always apply to software development.

Coordination burden due to scale, complexity, integrality of product, or a project team's geographic distribution—Some costs of reconfiguration in making software arise from the need to coordinate across different modules or organizational groups or geographical areas. Changing software in one place can create a costly problem in another place. These costs increase exponentially as the number of connections between people, places, and parts increases (Brooks 1995). Modular design (Baldwin and Clark 2000, MacCormack et al. 2007), which minimizes interdependencies among people, places, and parts, helps combat these costs. Modular design requires planning, however. Thus, this source of novelty cost suggests a need for plan-based design.

Context-dependent exploration costs—Even when reconfiguration costs are low, exploration costs—the cost of an outcome that “doesn't work out”—can still be high. Some contexts have a low tolerance for “bad” outcomes. The design of a making process will be seriously affected if expensive materials are lost or a plane crashes or someone dies when an outcome “doesn't work out.” Novelty costs increase as the tolerance for bad outcomes decreases. This factor in our contingency framework relates closely to the risk cited by Boehm and Turner (2003) as the determining factor in their contingency approach; they recommend planning when risk is high.

Implications for Future Research

We write this commentary in hopes that it might help us move past the current “strident” debates (Boehm and Turner 2003) about the design of software development and into a more nuanced discussion. The framework itself suggests directions for research that might help accomplish this.

More Nuanced Contingency Models

Grounded theory research aimed at a better understanding of the methods in use in particular circumstances, as characterized by general or specific sources of contingency, seems likely to be one helpful path forward. Inductive, qualitative studies suit inquiries into phenomena that seem, as evidenced by incompatible points of view and disagreements on fundamental issues, to be only roughly understood (Edmondson and McManus 2007). Case studies (Yin 1994) focused at the project level would probably be most feasible and prove most useful; they could provide rich insights and eventually yield testable hypotheses about questions that are more specific than “plan based or agile?” Inductive methods typically employ “theoretical sampling” (Eisenhardt 1989), choosing examples that span dimensions of interest; the contingency framework we have proposed contains useful sampling dimensions. For example, cases that vary across a range of novelty benefits (high to low) and novelty costs (high to low) might yield productive comparisons of development practices employed in these different contexts (HH, HL, LH, LL) and permit a look at the degree of success achieved by different practices in each context.

Sampling using specific sources of benefit and cost would also be useful. Projects for which the main source of novelty benefit is “potential for business differentiation” may differ in their development process design from projects that have access to novelty benefits because of “environmental turbulence.” Similarly, the scale of a development effort and geographic distribution of developers, both of which would influence the “coordination burden” source of novelty cost, might lead to different development process designs than those that arise in circumstances in which “context-dependent exploration costs” are the primary source of novelty cost.

We expect that studies like these could firmly establish that the choice between plan-based and agile

design approaches is not an either-or matter. We could well imagine, for example, a study of cases varying in “coordination burden” that shows that a plan-based approach, with lots of up-front design, can yield a sufficiently modular project organization (with low enough novelty cost) to make possible subsequent work using a more iterative approach. (As we have noted, the “product line” approach to software engineering provides an example that illustrates this idea; see Northrup et al. 2007.)

Such possibilities lead to testable hypotheses that are dynamic in nature. Plan-based approaches might create conditions in which later agile approaches could flourish. In different conditions, agile approaches might need to give way to plan-based approaches. Knowing the range of possible conditions, deriving such hypotheses, and establishing their validity would not only lead to further productive research, but could be a major advance for software development practices by suggesting ways of usefully combining plan-based and agile approaches.

Slaughter et al. (2006) provide a model for important aspects of such studies. They establish, through an inductive, case-based methodology, a relationship between (to use our terms) “potential for business differentiation” and process structure. Their study does not include novelty costs so it does not exemplify the sampling strategy we have proposed. Slaughter et al. (2006) acknowledge a need to address such costs in future research, and they present as theoretical background for their work the Hayes and Wheelwright (1979, 1984) product-process matrix, which has a clear cost basis along with its product variety differentiation basis. Despite its focus on a narrow category of novelty benefit, the article by Slaughter et al. (2006) remains a useful reference study; it demonstrates the rigor that qualitative research must exhibit to convince strongly opinionated scholars and practitioners. At the level of tactics, the study also provides examples of “concept maps” that can throw into relief the different development process details that arise in different conditions.

Using Contingency Frameworks to Address Other Research Questions

Our framework suggests other important directions of research. Accepting a contingency perspective on how to choose between plan-based and more iterative process structures does not require acceptance of

our more controversial hypothesis, which is that agile approaches arise now as part of a technology-enabled transition from industrial to post-industrial making. This hypothesis cries out for empirical investigation: Are conditions conducive to iterative development becoming more frequent in practice? The special conditions in which iterative (agile) approaches make sense, according to our framework, can only arise when *all* sources of novelty cost are low. What set of technologies must be assembled to handle novelty cost so completely?

Related to questions about the technologies required to enable iterative development process are questions about specific practices. As we have noted, "agile" is not just a matter of process structure but is also a set of closely associated values, principles, and practices (see, for example, "Manifesto for Agile Software Development," Beck et al. 2001). Some of these practices are aimed directly at accessing novelty benefits or reducing novelty costs. For example, the agile tendencies toward minimal documentation at least partly support the objective of reducing reconfiguration costs (by reducing the burden of document revision triggered by changes to software). The practices that involve close customer collaboration lead to novelty benefits that arise from the difficulty of discerning in advance what the customer needs from an intangible product. However, other ideas sometimes labeled "agile" are less simple to connect to novelty benefit or cost. Beck's insistence on a work week limited to 40 hours (Beck 1999) is an example. Agile proponents sometimes assert that their particular collections of methods must be applied in their entirety before the benefits of agility kick in. As Ågerfalk and Fitzgerald (2006) point out, this requirement seems pretty rigid for an approach that bills itself as a route to greater flexibility; and the assertion seems disproved by the variety of methods applied by those same proponents. Additional research could lead to better insight into this issue by revealing how specific methods enable more flexible process designs, and by examining whether alternative methods might work as well as those insisted on by agile proponents.

A contingency perspective also has implications for the conclusions we can draw from past research. It casts doubt on studies that compare approaches to development, but fail to control for differences

in the benefits from novelty or the cost of novelty inherent in the study context. Such concerns extend beyond software development. For instance, Cotteleer et al. (1998) describe an enterprise resource planning installation at Cisco Systems that used rapid, iterative prototyping, with considerable success. Additional research might suggest ways to reduce reconfiguration and exploration costs in nondevelopment projects. Clearly, sensitivity to the inherent benefits and costs of novelty must extend to these domains if we are to get a valid sense of what works and what does not.

If the appearance of agile approaches does, as we have suggested, herald a transition from industrial to post-industrial making at least for some productive activities, then we might also gain from studying institutional factors likely to facilitate or hinder the transition. Economist Paul David, in a study of how manufacturers adopted electric motors to replace steam engines as sources of power, shows that such transitions are usually accompanied by institutional friction (David 1990). It took the better part of a generation for the advantages offered by electric motors to be realized because they were at first inserted into processes, systems, organizations, and institutions designed for steam engines. In conversation, a corporate chief information officer (CIO) recently told us a parallel story: His efforts to organize software development iteratively in his organization had run afoul of standards for capitalizing software investment. The problem? The standards assumed a waterfall development model.

Conclusion

Whether or not you believe that an historic transition is associated with iterative development approaches, there is little doubt that information technology (and software, in particular) will continue to influence how we make things, probably in increasingly dramatic ways. In drug research, companies increasingly use robots to conduct thousands of experiments in parallel, cheaply and rapidly (Thomke 2003). Architects and designers use "computer aided virtual environments" (CAVEs) to "experience" buildings and other products as virtual prototypes. Architect William J. Mitchell notes that "digital technology allows architects to reduce reliance on standardization and repetition and

to produce designs that respond more sensitively to varied conditions and needs” (Mitchell 2004); he offers as an example the Stata Center at MIT, a building of a structural complexity that would not have been attempted in an earlier time. These possibilities make it all the more important for us to move beyond entrenched disagreements about planning versus agility to a nuanced, research-based, and practice-relevant conversation that explores the contingencies involved in choosing an appropriate software development process design.

Acknowledgments

The authors thank the Division of Research at the Harvard Business School, the President of Copenhagen Business School, and the President of Swarthmore College for support of this research. The authors also thank conference and journal reviewers, Kent Beck, Jim Highsmith, and colleagues at Harvard Business School, the C. T. Bauer College of Business at the University of Houston, the Graduate School of Business at the University of Washington, the McIntire School of Commerce at the University of Virginia, and Microsoft Corporation, all of whom provided useful feedback on earlier versions of this paper.

References

- Agerfalk, P. J., B. Fitzgerald. 2006. Flexible and distributed software processes: Old petunias in new bowls? *Comm. ACM* 49(10) 27–34.
- Amabile, T. 1996. *Creativity in Context*. Westview Press, Boulder, CO.
- Austin, R. D. 2001. Ford Motor Company: Supply chain strategy. Harvard Business School Case 699-198, Harvard Business School Publishing, Boston.
- Austin, R. D. 2008. High margins and the search for aesthetic coherence. *Harvard Bus. Rev.* 86(1) 18–19.
- Austin, R. D., L. Devin. 2003. *Artful Making: What Managers Need to Know About How Artists Work*. Financial Times Prentice Hall, Upper Saddle River, NJ.
- Baldwin, C. Y., K. B. Clark. 2000. *Design Rules: The Power of Modularity*. MIT Press, Cambridge, MA.
- Banker, R., C. Kemerer. 1989. Scale economies in new software development. *IEEE Trans. Software Engrg.* 15(10) 1199–1205.
- Banker, R., S. Slaughter. 1997. A field study of scale economies in software maintenance. *Management Sci.* 43(12) 1709–1725.
- Beck, K. 1999. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston.
- Beck, K., M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, et al. 2001. Manifesto for agile software development, Accessed January 25, 2008, <http://agilemanifesto.org/>.
- Boehm, B. W. 1988. A spiral model of software development and enhancement. *IEEE Comput.* 21(5) 61–72.
- Boehm, B. W., R. Turner. 2003. *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley Professional, Boston.
- Bohm, C., G. Jacopini. 1966. Flow-diagrams, turing machines, and languages with only two formation rules. *Comm. ACM* 9 366–371.
- Brooks, Jr., F. P. 1995. *The Mythical Man-Month: Essays on Software Engineering: Anniversary Edition*. Addison-Wesley, Reading, MA.
- Brooks, Jr., F. P. 1987. No silver bullet: Essence and accidents of software engineering. *IEEE Comput.* 20(4) 10–19.
- Christensen, C. M. 1997. *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail*. Harvard Business School Press, Boston.
- Clark, K., T. Fujimoto. 1991. *Product Development Performance: Strategy, Organization, and Performance in the World Auto Industry*. Harvard Business School Press, Boston.
- Conboy, K. 2009. Agility from first principles: Reconstructing the concept of agility in information systems development. *Inform. Systems Res.* 20(3) 329–354.
- Cooper, R. G. 1986. *Winning at New Products*. Holt, Rinehart, Winston, New York.
- Cooper, R. G. 1988. The new product process: A decision guide for managers. *J. Marketing Management* 3(3) 238–255.
- Cotteleur, M. J., R. D. Austin, R. L. Nolan. 1998. Cisco Systems, Inc.: Implementing ERP. Case No. 699-022, Harvard Business School, Harvard Business School Publishing, Boston.
- Cox, B. J. 1990. Planning the software industrial revolution. *IEEE Software* 7(6) 25–33.
- Dahl, O. J., K. Nygaard. 1967. SIMULA '67 common base proposal. Norwegian Computing Center, Oslo, Norway.
- Dahl, O. J., E. W. Dijkstra, C. A. R. Hoare. 1972. *Structured Programming*. Academic Press, London and New York.
- David, P. A. 1990. The dynamo and the computer: A historical perspective on the modern productivity paradox. *Amer. Econom. Rev.* 80(2) 355–361.
- Dijkstra, E. W. 1968. Go to statement considered harmful. *Comm. ACM* 11(3) 147–148.
- Dijkstra, E. W. 1969. Notes on structured programming (circulated manuscript). Accessed January 25, 2008, <http://www.cs.utexas.edu/users/EWD/ewd02xx/EWD249.PDF>.
- Edmondson, A. C., S. E. McManus. 2007. Methodological fit in management field research. *Acad. Management Rev.* 32(4) 1155–1179.
- Eisenhardt, K. M. 1989. Building theories from case study research. *Acad. Management Rev.* 14(4) 532–550.
- Eisenhardt, K. M., B. N. Tabrizi. 1995. Accelerated adaptive processes: Product innovation in the global computer industry. *Admin. Sci. Quart.* 40(1) 84–110.
- Ford, H. 1922. *My Life and Work*. Garden City Publishing, Garden City, NY.
- Franke, N., E. Von Hippel. 2003. Satisfying heterogeneous needs via innovation toolkits: The case of Apache security software. *Res. Policy* 32(7) 1199–1215.
- Fröberg, J. 2002. Software components and COTS in software system development. I. Crnkovic, M. Larsson, eds. *Building Reliable Component-Based Software Systems*. Artech House, Boston.
- George, J. F. 2000. The origins of software: Acquiring systems at the end of the century. R. W. Zmud, ed. *Framing the Domains of IT Management Research: Glimpsing the Future Through the Past*. Pinnaflex Educational Resources, Cincinnati.
- Harris, M., R. Collins, A. Hevner. 2009. Control of flexible software development under uncertainty. *Inform. Systems Res.* 20(3) 400–419.

- Hayes, R., S. Wheelwright. 1979. Link manufacturing process and product life cycles. *Harvard Bus. Rev.* 57(1) 133–140.
- Hayes, R., S. Wheelwright. 1984. *Restoring Our Competitive Edge: Competing Through Manufacturing*. Wiley, New York.
- Highsmith, III, J. H. 1999. *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems*. Dorset House, New York.
- Highsmith III, J. H. 2002. *Agile Software Development Ecosystems*. Addison-Wesley, Boston.
- Hopp, W. J., M. L. Spearman. 2004. To pull or not to pull: What is the question? *Manufacturing Service Oper. Management* 6(2) 133–148.
- Iansiti, M., A. MacCormack. 1996. Team New Zealand (A). Case 9-696-040 Harvard Business School, Harvard Business School Publishing, Boston.
- Iansiti, M., A. MacCormack. 1997. Developing products on Internet time. *Harvard Bus. Rev.* 75(5) 108–117.
- Jacobson, I., G. Booch, J. Rumbaugh. 1999. *The Unified Software Development Process*. Addison-Wesley, Boston.
- Kanigel, R. 1997. *The One Best Way: Frederick Winslow Taylor and the Enigma of Efficiency*. Viking, New York.
- Kay, A. C. 1993. The early history of smalltalk. *ACM SIGPLAN Notices* 28(3) 69–95.
- Keller, C. M., J. Dixon Keller. 1996. *Cognition and Tool Use: The Blacksmith at Work*. Cambridge University Press, Cambridge, UK.
- Kemerer, C. 1992. How the learning curve affects case tool adoption. *IEEE Software* 9(3) 23–28.
- Knuth, D. 1974. Structured programming with goto statements. *Comput. Surveys* 6(4) 261–301.
- Kracik, J. F. 1988. Triumph of the lean production system. *MIT Sloan Management Rev.* 30(1) 41–52.
- MacCormack, A. D., R. Verganti, M. Iansiti. 2001. Developing products on Internet time: The anatomy of a flexible development process. *Management Sci.* 47(1) 133–150.
- MacCormack, A., J. Rusnak, C. Y. Baldwin. 2007. The impact of component modularity on design evolution: Evidence from the software industry. Unit Research Paper 08-038, Harvard Business School Technology & Operations Management, <http://ssrn.com/abstract=1071720>.
- MacDuffie, J. P. 1995. Human resource bundles and manufacturing performance: Organizational logic and flexible production systems in the world auto industry. *Indust. Labor Relations Rev.* 48(2) 197–221.
- Martin, J. 1991. *Rapid Application Development*. McMillan, New York.
- Marwaha, S., S. Patil, R. Tinaikar. 2006. The next generation of in-house software development. *McKinsey Quart. Online J. McKinsey & Co* (February).
- McCalley, B. W. 1994. *Model T Ford: The Car That Changed the World*. Krause Publications, Iola, WI.
- Mills, H. 1971. Top-down programming in large systems. R. Ruskin, ed. *Debugging Techniques in Large Systems*. Prentice-Hall, Englewood Cliffs, NJ.
- Mirsky, J., A. Nevins. 1952. *The World of Eli Whitney*. The Macmillan Company, New York.
- Mitchell, W. J. 2004. Constructing complexity in the Digital Age. *Science* 303(5) 1472–1474.
- Nambisan, S. 2003. Information systems as a reference discipline for new product development. *MIS Quart.* 27(1) 1–18.
- Nevins, A. 1954. *Ford: The Man, the Company*. With the collaboration of F. F. Hill. Charles Scribner and Sons, New York.
- Northrop, L. M., P. C. Clements, F. Bachmann, J. Bergey, G. Chastek, S. Cohen, P. Donohoe, et al. 2007. A framework for software product line practice. Working paper, Version 5.0, Software Engineering Institute, Carnegie Mellon University, Pittsburgh (July).
- Osterweil, L. 1987. Software processes are software too. *Proc. 9th Internat. Conf. Software Engrg.*, Monterey, CA (March), 2–13.
- Parnas, D. 2006. Agile methods and GSD: The wrong solution to an old but real problem. *Comm. ACM* 49(10) 29.
- Paulk, M. C. 2002. Agile methodologies and process discipline. *Crosstalk: J. Defense Software Engrg.* Accessed January 25, 2008, <http://www.stsc.hill.af.mil/crosstalk/2002/10/paulk.html>.
- Royce, W. W. 1970. Managing the development of large software systems: Concepts and techniques. *Proc.*, WESCON (August).
- Sannella, D. 2008. Algebraic and logical foundations of formal software development. Accessed January 25, 2008, <http://homepages.inf.ed.ac.uk/dts/alf/>.
- Sannella, D., A. Tarlecki. 1997. Essential concepts of algebraic specification and program development. *Formal Aspects Comput.* 9 229–269.
- Schrage, M. 2000. *Serious Play: How the World's Best Companies Simulate to Innovate*. Harvard Business School Press, Boston.
- Shah, R., P. T. Ward. 2003. Lean manufacturing: Context, practice bundles, and performance. *J. Oper. Management* 21(2) 129–149.
- Slaughter, S. A., D. Harter, M. Krishnan. 1998. Evaluating the cost of software quality. *Comm. ACM* 41(8) 67–73.
- Slaughter, S. A., L. Levine, R., Balasubramaniam, J. Pries-Heje, R. Baskerville. 2006. Aligning software processes with strategy. *MIS Quart.* 30(4) 891–918.
- Spear, S., H. K. Bowen. 1999. Decoding the DNA of the Toyota production system. *Harvard Bus. Rev.* 77(5) 97–106.
- Staats, B. R., D. M. Upton. 2007. Lean principles, learning, and software production: Evidence from Indian software services. Working Paper 08-001, Harvard Business School, Boston.
- Thomke, S. 2003. *Experimentation Matters*. Harvard Business School Press, Boston.
- Voas, J. 1998. COTS software: The economical choice? *IEEE Software* 15(2) 16–19.
- Wirth, N., J. Gutknecht. 1992. Ten years after: From objects to components. Project Oberon: Design of an Operating System and Compiler. Addison-Wesley, Boston, p. 434, 2005 (original 1992), <http://www.oberon.ethz.ch/WirthPubl/ProjectOberon.pdf>.
- Womack, J. P., D. T. Jones, D. Roos. 1990. *The Machine That Changed the World: The Story of Lean Production*. Rawson Associates, New York.
- Wood, J., D. Silver. 1995. *Joint Application Design*, 2nd ed. John Wiley & Sons, New York.
- Yin, R. 1994. *Case Study Research: Design and Methods*, 2nd ed. Sage Publications, Thousand Oaks, CA.